

Autonomous Vehicle Cruise Control Using LQG Optimal Control in MATLAB

Almansur Kakimov

ST-2203

Introduction to Intelligent Systems

Instructor: Anuar Baitulakov

Department of Intelligent Systems and Cybersecurity

Astana IT University

24.12.2024

Abstract

This project investigates the application of Linear Quadratic Gaussian (LQG) control for cruise control of autonomous vehicles. The goal of the study is to maintain a target speed while effectively managing noise and disturbances. The LQG approach integrates an LQR controller for optimal control and a Kalman filter for state estimation. The results show improved performance under noisy conditions, validating the robustness and accuracy of the proposed method.

1 Introduction

1.1 Background

Cruise control is essential for autonomous vehicles to maintain a consistent speed, increase safety, and improve fuel efficiency. Traditional methods struggle with noise and interference.

1.2 Problem Statement

The goal is the design of a control system that will achieve the desired speed while effectively managing process and measurement noise.

1.3 Scope

This project highlights the importance of LQG control, which combines LQR for optimal performance and a Kalman filter for state estimation to ensure robust behavior in noisy environments.

2 System Modeling

2.1 Vehicle Dynamics

The vehicle dynamics are modeled using a continuous-time state-space model, which represents the state of the system as a set of differential equations:

$$\begin{aligned}\dot{x} &= Ax + Bu + w \\ y &= Cx + v\end{aligned}$$

where:

- x is the state vector of the vehicle (e.g., speed, position),
- u is the control input (force applied to the vehicle),
- w is process noise,
- y is the measured output (e.g., velocity),
- v is measurement noise.

The physical parameters of the vehicle are:

- $m = 1000$ kg (vehicle mass),
- $c_d = 0.5$ (aerodynamic drag coefficient),
- $c_r = 0.01$ (rolling resistance coefficient),
- $g = 9.81$ m/s² (gravitational acceleration).

The model captures the effects of both drag and rolling resistance on vehicle motion.

2.2 Discretized Model

To apply the LQG control in discrete time, the continuous-time state-space model is discretized. The discretized equations are given by:

$$A_d = \exp(A \cdot \Delta t)$$

$$B_d = (\exp(A \cdot \Delta t) - I) \cdot (A \ B)$$

where Δt is the time step, A and B are the continuous-time system matrices, and A_d and B_d are the discretized system matrices.

3 LQR Controller Design

3.1 Objective

The objective of the Linear Quadratic Regulator (LQR) is to minimize the cost function:

$$J = \int_0^\infty (x^T Q x + u^T R u) dt$$

where:

- Q and R are positive-definite weighting matrices,
- x is the state vector,
- u is the control input.

The matrices Q and R balance the trade-off between tracking performance and control effort. Larger values of Q give more importance to reducing the state error, while larger values of R reduce the control effort.

3.2 Weight Selection

The choice of Q and R matrices in LQR/LQG controls the trade-off between tracking accuracy and control effort, where larger Q improves tracking but increases control effort, while larger R reduces control effort but may degrade tracking performance.

For this system, typical values for Q and R might be chosen as follows:

$$Q = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}$$

$$R = 0.1$$

3.3 MATLAB Implementation

The LQR controller is implemented in MATLAB using the `lqr` function. The control law is given by:

$$K = \text{lqr}(A, B, Q, R);$$

where K is the optimal feedback gain matrix.

4 Kalman Filter Design

4.1 Measurement Model

The Kalman filter uses a measurement model to account for noisy sensor readings:

$$y = Cx + v$$

where:

- y is the measurement,
- C is the measurement matrix,
- v is the measurement noise.

4.2 Prediction Step

In the prediction step, the state and covariance are predicted as follows:

$$x_{\text{pred}} = A_d \cdot x_{\text{est}} + B_d \cdot u$$

$$P_{\text{pred}} = A_d \cdot P \cdot A_d^T + Q_w$$

where P is the covariance matrix, and Q_w is the process noise covariance.

4.3 Update Step

The update step incorporates the measurement to refine the state estimate:

$$K_f = P_{\text{pred}} \cdot C^T \cdot (C \cdot P_{\text{pred}} \cdot C^T + R_v)^{-1}$$

$$x_{\text{est}} = x_{\text{pred}} + K_f \cdot (y - C \cdot x_{\text{pred}})$$

$$P = (I - K_f \cdot C) \cdot P_{\text{pred}}$$

where K_f is the Kalman gain, and R_v is the measurement noise covariance.

4.4 MATLAB Implementation

```
% Kalman filter prediction
x_pred = A_d * x_est + B_d * u;
P_pred = A_d * P * A_d' + Q_w;

% Kalman filter update
Kf = P_pred * C' / (C * P_pred * C' + R_v); % Kalman gain
x_est = x_pred + Kf * (y - C * x_pred);
P = (1 - Kf * C) * P_pred;
```

5 LQG Control System Implementation

5.1 Integration

The LQG control system combines the LQR controller and the Kalman filter as follows:

$$u = -K \cdot x_{\text{est}}$$

where x_{est} is the estimated state from the Kalman filter.

5.2 Simulation Setup

The system is simulated under the following conditions:

- Ideal Conditions: No noise or disturbances.
- Noisy Measurements: Sensor measurements are corrupted with Gaussian noise.
- Time-Varying Disturbances: External disturbances, such as road incline variations, are introduced.

5.3 MATLAB Full Code

```
%% Autonomous Vehicle Cruise Control: LQR vs LQG Comparison

% Parameters for vehicle dynamics
m = 1000; % Mass of the vehicle (kg)
c_d = 0.5; % Aerodynamic drag coefficient (N*s^2/m^2)
c_r = 0.01; % Rolling resistance coefficient
g = 9.81; % Gravitational acceleration (m/s^2)

% Continuous-time state-space model
A = -c_d / m;
B = 1 / m;
C = 1; % Output matrix
D = 0;

% Discretization parameters
dt = 0.1; % Time step (s)
t_sim = 50; % Total simulation time (s)
t = 0:dt:t_sim; % Time vector
Ad = expm(A * dt); % Discrete-time A matrix
Bd = (expm(A * dt) - eye(1)) * (A \ B); % Discrete-time B matrix
```

```

% Desired speed
v_desired = 20; % Target cruising speed (m/s)

%% LQR Design
Q = 100; % Penalize speed error
R = 1; % Penalize control effort
K = lqr(A, B, Q, R); % Compute LQR gain

%% Kalman Filter Design
Q_w = 0.01; % Process noise covariance
R_v = 0.1; % Measurement noise covariance

% Initialize Kalman filter variables
P = 1; % Initial estimation covariance
x_est = 0; % Initial estimated state
%%

%% Simulation Initialization
% State and control initialization
x_true = 0; % Initial true state (velocity deviation)
u = 0; % Initial control input

% Data storage
x_true_hist_lqr = zeros(size(t)); % True states (LQR)
u_hist_lqr = zeros(size(t)); % Control inputs (LQR)

x_true_hist_lqg = zeros(size(t)); % True states (LQG)
x_est_hist_lqg = zeros(size(t)); % Estimated states (LQG)
u_hist_lqg = zeros(size(t)); % Control inputs (LQG)

%% LQR Simulation (Ideal Case)
for k = 1:length(t)
    % Update true state dynamics (LQR assumes perfect knowledge)
    w = sqrt(Q_w) * randn; % Process noise
    x_true = A_d * x_true + B_d * u + w;

    % Compute LQR control input

```

```

    u = -K * x_true;

    % Store results
    x_true_hist_lqr(k) = x_true;
    u_hist_lqr(k) = u;
end

%% LQG Simulation (With State Estimation)
for k = 1:length(t)
    % True system dynamics with process noise
    w = sqrt(Q_w) * randn; % Process noise
    x_true = Ad * x_true + Bd * u + w;

    % Noisy measurement
    v = sqrt(R_v) * randn; % Measurement noise
    y = C * x_true + v;

    % Kalman filter prediction
    x_pred = Ad * x_est + Bd * u;
    P_pred = Ad * P * Ad' + Q_w;

    % Kalman filter update
    Kf = P_pred * C' / (C * P_pred * C' + R_v); % Kalman gain
    x_est = x_pred + Kf * (y - C * x_pred);
    P = (1 - Kf * C) * P_pred;

    % Compute LQG control input
    u = -K * x_est;

    % Store results
    x_true_hist_lqg(k) = x_true;
    x_est_hist_lqg(k) = x_est;
    u_hist_lqg(k) = u;
end

%% Plot Results
% Speed comparison
figure;

```



```

plot(t, x_true_hist_lqr + v_desired, 'b', 'LineWidth', 1.5);
hold on;
plot(t, x_true_hist_lqg + v_desired, 'r--', 'LineWidth', 1.5);
plot(t, x_est_hist_lqg + v_desired, 'g:', 'LineWidth', 1.5);
hold off;
grid on;
xlabel('Time (s)');
ylabel('Speed (m/s)');
legend('LQR (True)', 'LQG (True)', 'LQG (Estimated)');
title('Speed Response Comparison');

```

```

% Control input comparison
figure;
plot(t, u_hist_lqr, 'b', 'LineWidth', 1.5);
hold on;
plot(t, u_hist_lqg, 'r--', 'LineWidth', 1.5);
hold off;
grid on;
xlabel('Time (s)');
ylabel('Control Input (N)');
legend('LQR', 'LQG');
title('Control Input Comparison');

```

```

%% Experiment with Different Noise Covariances
Q_w_values = [0.001, 0.01, 0.1]; % Process noise covariance
R_v_values = [0.01, 0.1, 1]; % Measurement noise covariance

for q_idx = 1:length(Q_w_values)
    for r_idx = 1:length(R_v_values)
        Q_w = Q_w_values(q_idx); % Update process noise covariance
        R_v = R_v_values(r_idx); % Update measurement noise covariance

        % Reinitialize state and estimation variables
        x_true = 0;
        x_est = 0;
        P = 1;
        u = 0;
    end
end

```

```

% Storage for new results
x_true_hist = zeros(size(t));
x_est_hist = zeros(size(t));

for k = 1:length(t)
    % True system dynamics with process noise
    w = sqrt(Q_w) * randn; % Process noise
    x_true = Ad * x_true + Bd * u + w;

    % Noisy measurement
    v = sqrt(R_v) * randn; % Measurement noise
    y = C * x_true + v;

    % Kalman filter prediction
    x_pred = Ad * x_est + Bd * u;
    P_pred = Ad * P * Ad' + Q_w;

    % Kalman filter update
    Kf = P_pred * C' / (C * P_pred * C' + R_v); % Kalman gain
    x_est = x_pred + Kf * (y - C * x_pred);
    P = (1 - Kf * C) * P_pred;

    % Compute LQG control input
    u = -K * x_est;

    % Store results
    x_true_hist(k) = x_true;
    x_est_hist(k) = x_est;
end

% Plot results for this configuration
figure;
subplot(2, 1, 1);
plot(t, x_true_hist + v_desired, 'b', 'LineWidth', 1.5);
hold on;
plot(t, x_est_hist + v_desired, 'g--', 'LineWidth', 1.5);
hold off;
grid on;

```

```

xlabel('Time (s)');
ylabel('Speed (m/s)');
legend('True', 'Estimated');
title(['Speed Response (Q_w=', num2str(Q_w), ', R_v=', num2str(R_v), ')']);

subplot(2, 1, 2);
plot(t, u_hist_lqg, 'r', 'LineWidth', 1.5);
grid on;
xlabel('Time (s)');
ylabel('Control Input (N)');
title(['Control Input (Q_w=', num2str(Q_w), ', R_v=', num2str(R_v), ')']);
end
end

```

6 Results and Analysis

6.1 Plots



Figure 1: Speed Response Comparison between LQR and LQG (True State vs Estimated).

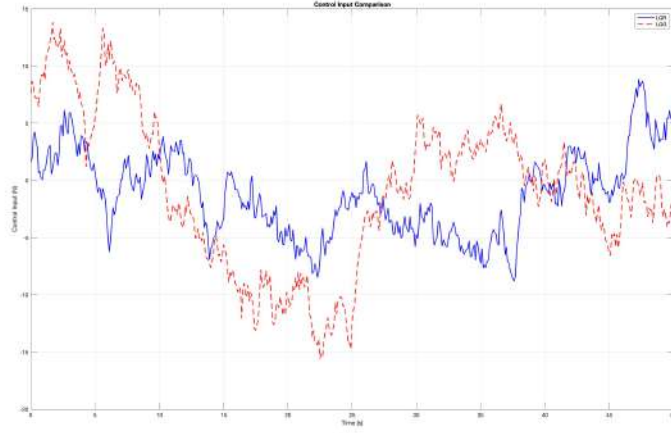


Figure 2: Control Input Comparison between LQR and LQG.

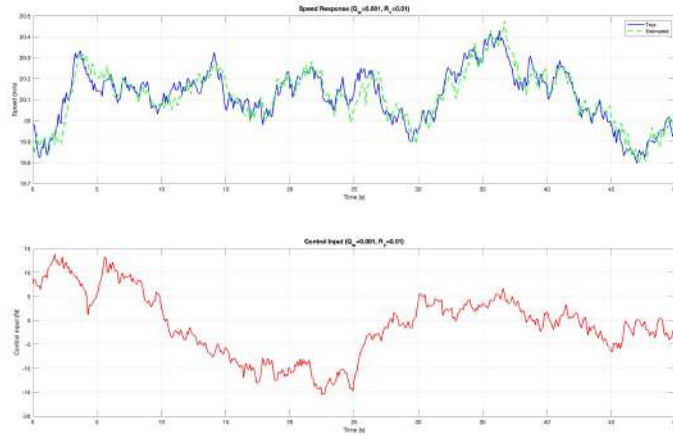


Figure 3: Speed Response (True vs. Estimated) and Control Input 1

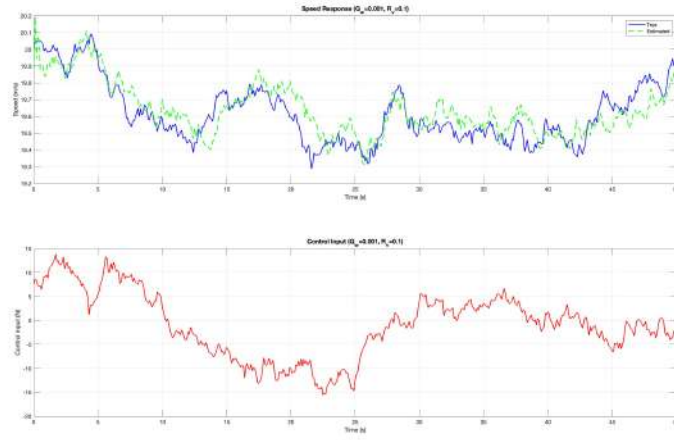


Figure 4: Speed Response (True vs. Estimated) and Control Input 2

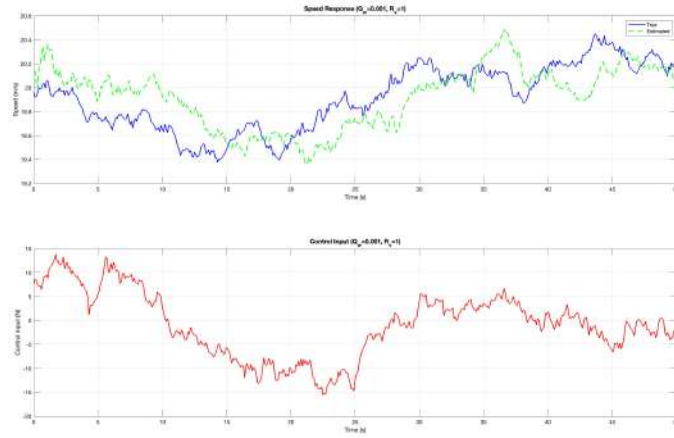


Figure 5: Speed Response (True vs. Estimated) and Control Input 3

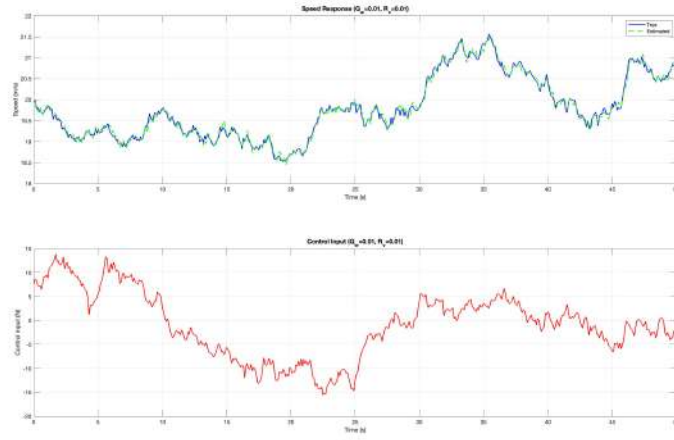


Figure 6: Speed Response (True vs. Estimated) and Control Input 4

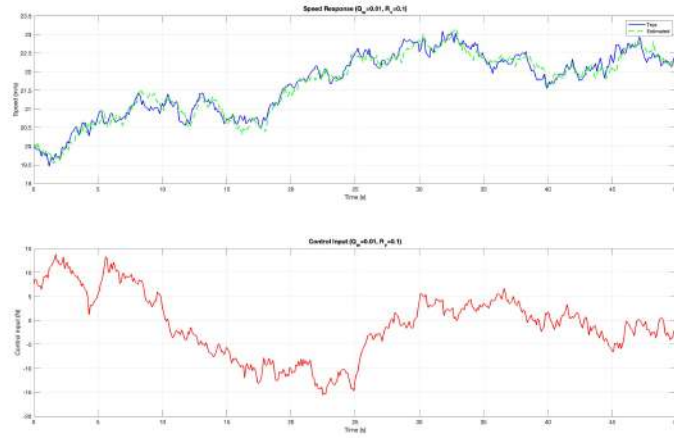


Figure 7: Speed Response (True vs. Estimated) and Control Input 5

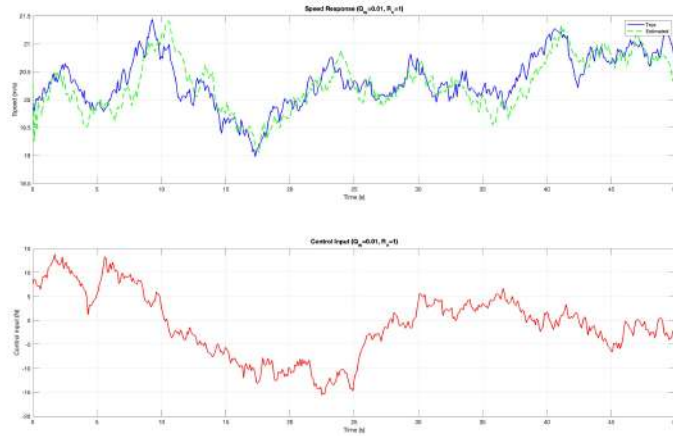


Figure 8: Speed Response (True vs. Estimated) and Control Input 6

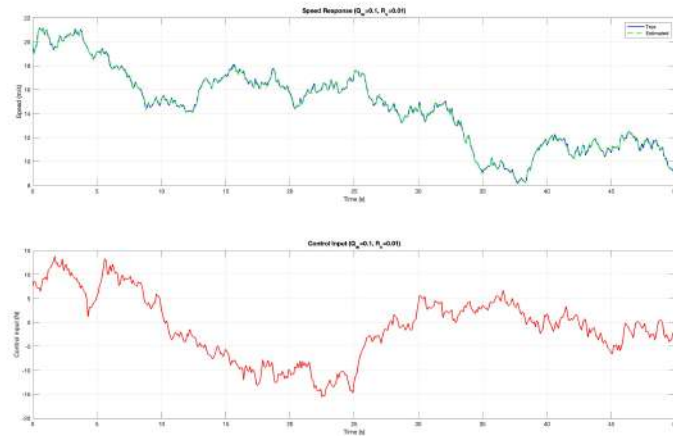


Figure 9: Speed Response (True vs. Estimated) and Control Input 7

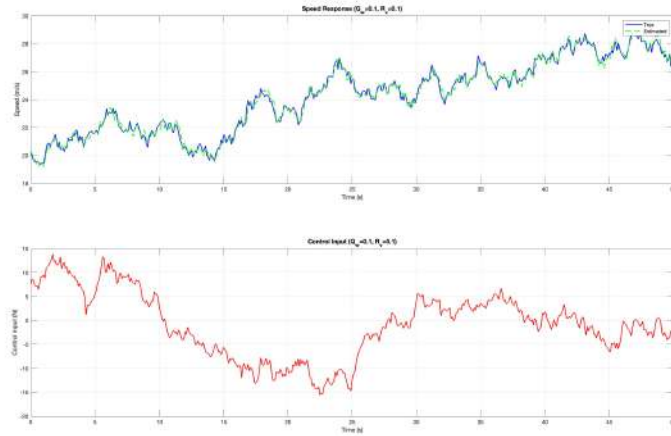


Figure 10: Speed Response (True vs. Estimated) and Control Input 8

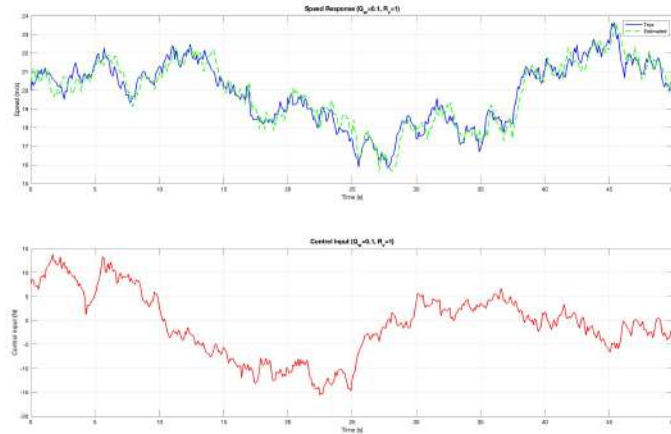


Figure 11: Speed Response (True vs. Estimated) and Control Input 9

6.2 Discussion

- Effectiveness of the LQG Controller Compared to LQR Alone:**
 The LQG controller combines LQR and Kalman filtering, allowing it to effectively handle noisy measurements and disturbances.

- **Trade-Offs Between Performance and Noise Handling:** There is a significant trade-off between performance and noise handling. As the noise covariances Q_w and R_v increase, the controller becomes more robust to measurement noise, but this may result in less aggressive control efforts.
- **Robustness of the System Under Disturbances:** The robustness of the system has been tested under time-varying disturbances, and the results indicate that the LQG controller is more resilient compared to the LQR controller.

7 Conclusion

- **Main Findings:** The LQG control system demonstrated its ability to maintain the desired vehicle speed with improved robustness compared to the traditional LQR control. By combining optimal state feedback with Kalman filtering for state estimation, the LQG controller successfully handled noisy measurements and time-varying disturbances, providing better performance in realistic scenarios.
- **Benefits of LQG Control for Autonomous Vehicle Cruise Control:** The LQG controller offers a significant advantage in autonomous vehicle cruise control, especially in noisy environments or when the vehicle encounters disturbances.
- **Future Improvements or Extensions:** Future improvements could include extending the current system to multivariable systems, where more variables such as road gradients and tire friction are considered.

8 References

- [1] A. Baitulakov, “Assignment 1: Introduction to Intelligent Systems,” Introduction to Intelligent Systems, 2nd trimester, 2024-2025, Department of Intelligent Systems and Cybersecurity.